Open in Colab

Functions

In Python functions are objects that can be called with some parameters and return some object. Then you can use that object in your program.

There are many builtin functions defined in Python.

Using Functions

In [1]:
```
list1 = [4, 2, 8, 11]
length = len(list1)
print("length of the list1 is:", length)
```

```
length of the list1 is: 4
```

In [2]:
```
print("the maximum of the list1 is:", max(list1))
```

```
the maximum of the list1 is: 11
```

`print` is another builtin Python function:

In [3]:
```
a = print("hello world!")
```

```
hello world!
```

Warning: Some functions seems not returning anything. But any Python function **returns** some object.

In [4]:
```
print(a)
```

```
None
```

Here `print` function returned a special object: `None`

Also some Python can be called without any parameters:

In [5]:
```
tuple()
```

Out[5]:
```
()
```

Even if you don't give any parameters you must use paranthesis in order to call the function. If you write just the name of the function then the function itself is returned not the object returned by the function. So the following code does not evaluates the function.

In [6]:
```
myfunction = tuple
```

Defining New Functions

One way to define a function is to use

- `def` keyword followed by
- a valid variable name,
- a pair of paranthesis and

- `:`

Like `if` , , `while` and `for` statements, after `:` you must put some indentation if you want to continue with the next line, or write a single statement after `:`

In [7]:
```
def isodd(x): return x & 1
```

In [8]:
```
isodd(18)
```

Out[8]:
```
0
```

Any python statement can be written inside function definiton block.

In [9]:
```
def my_function(): print("hello world!")
```

In [10]:
```
my_function()
```

```
hello world!
```

In [11]:
```
def gcd(a,b):
  "This function calculates the Greatest Common Divisor of given numbers in ancient wa
  while a != b:
    if a > b:
      a -= b
    if a < b:
      b -= a

  return a
```

In [12]:
```
x = gcd(15,21)
print(x)
```

```
3
```

The first string in function definition block is called **doc string** and it is used to help people understand your function.

In [13]:
```
help(gcd)
```

```
Help on function gcd in module __main__:

gcd(a, b)
    This function calculates the Greatest Common Divisor of given numbers in ancient w
ay.
```

In [14]:
```
help(print)
```

```
Help on built-in function print in module builtins:

print(*args, sep=' ', end='\n', file=None, flush=False)
    Prints the values to a stream, or to sys.stdout by default.

    sep
      string inserted between values, default a space.
    end
      string appended after the last value, default a newline.
    file
      a file-like object (stream); defaults to the current sys.stdout.
    flush
      whether to forcibly flush the stream.
```

In function definiton, `return` does two things:

- terminates the function execution and
- returns a value.

You can use multiple `return` statements in a function definition. But only the first evaluated will be effective. Because `return` also ends the execution of the function.

In [15]:
```
def gcd(a,b):
  "this function calculates the GCD of given numbers in ancient way."

  if a <= 0 or b <= 0:
    print("This version of gcd algorith works only for non-negative integers!")
    return

  while a != b:
    if a > b:
      a -= b
    if a < b:
      b -= a

  return a
```

In [16]:
```
print(gcd(15,-1))
```

```
This version of gcd algorith works only for non-negative integers!
None
```

In [17]:
```
gcd(1.5,2.25)
```

Out[17]:
```
0.75
```

Here `return` means `return None`. Usually it is used when the function needs to be terminated without returning any useful value.

If you ommit the `return` statement when defining a function then Python automatically returns `None` after reaching to the end of the block.

In [18]:
```
def dummy(a,b):
  "a dummy message"
  a + b
```

Here the function `dummy` adds `a` and `b` but since it doesn't have a `return` statement it returns `None` instead of `a+b`.

In [19]:
```
print(dummy(3,4))
```

```
None
```

Accessing Variables

You can use a variable defined outside of a function as usual.

In [20]:
```
a = 0
```

In [21]:
```
def f():
    print(a)
```

This function `f()` prints the value of the variable `a` when called.

In [22]:
```
f()
```

```
0
```

But if you want to change the value of a variable defined outside of the function by **assigning** it another value that change is done only **locally**.

In [23]:
```
def g():
    a = 12
    print(a)
```

This function `g()` creates a new variable `a` with value `5` and prints its value.

In [24]:
```
g()
```

```
12
```

The variable `a` defined outside of the function is not altered.

In [25]:
```
a
```

Out[25]:
```
0
```

To be able to change the value of a variable inside a function use `global` statement.

In [26]:
```
def h():
    global a
    a = 25
```

In [27]:
```
h()
```

Now the value of the variable defined before is changed.

In [28]:
```
a
```

Out[28]:
```
25
```

Arguments

Positional vs. Keyword Arguments

Normally, you must write the arguments of the function in order. These are called **positional** arguments. For example:

In [29]:
```
def my_function2(alice, bob):
  print(alice, "has a message to:", bob)
```

In [30]:
```
my_function2("Ayşe","Ali")
```

```
Ayşe has a message to: Ali
```

is different than:

In [31]:
```
my_function2("Ali","Ayşe")
```

```
Ali has a message to: Ayşe
```

But in Python you can also pass the arguments by their names. So the order becomes unimportant. These arguments are called **keyword** arguments.

In [32]:
```
my_function2(bob="Ali",alice="Ayşe")
```

```
Ayşe has a message to: Ali
```

Warning: You can not pass positional arguments after any keyword argument.

In [33]:
```
my_function2("Ayşe",bob="Ali")
```

```
Ayşe has a message to: Ali
```

If you want to define a function with some positional only arguments use `/` to seperate them from other arguments.

In [34]:
```
def gcd(a,b,/):
  "this function calculates the GCD of given numbers in a modern way."
  while b:
    a, b = b, a%b
  return a
```

In [35]:
```
gcd(12,56)
```

Out[35]:
```
4
```

If you want to define a function with some keyword only arguments use `*` to seperate them from other arguments.

In [36]:
```
def fibonacci(n,/,*,a,b):
  """
  this function finds the largest fibonacci number little then the given number.
  optionally you can specify initial values of a and b.
  """
  while b < n:
    a, b = b, a+b
  return b
```

In [37]:
```
fibonacci(100,a=0,b=1)
```

Out[37]:
```
144
```

Optional Arguments

When defining a function you can give default values to some arguments.

In [38]:
```
def fibonacci(n,a=0,b=1):
  """
  this function finds the largest fibonacci number little then the given number.
  optionally you can specify initial values of a and b.
  """
  while b < n:
    a, b = b, a+b
  return a
```

In [39]:
```
fibonacci(100)
```

Out[39]:
```
89
```

In [40]:
```
fibonacci(100,a=1,b=3)
```

Out[40]:
```
76
```

In [41]:
```
tuple("hello")
```

```
# help(tuple)
```

Out[41]:
```
('h', 'e', 'l', 'l', 'o')
```

In [42]:
```
def int_exp(exp, base=10):
  result = 1
  for i in range(exp):
    result *= base
  return result
```

Here the argument `base` is optional. If you don't give a value it will be regarded as `10`.

In [43]:
```
int_exp(6), int_exp(6,2)
```

Out[43]:
```
(1000000, 64)
```

Warning: When defining a function, all the non-optional arguments must be before optional arguments.

Variable Number of Arguments

Unpacking Iterable Arguments

Like assignment you can use `*` unpack an iterable to pass them to multiple arguments. Otherwise Python will assume that a single object.

In [44]:
```
pair = 4, 2
int_exp(pair[0],pair[1])
```

Out[44]:
```
16
```

In [45]:
```
pair = 4, 2
int_exp(*pair)
```

Out[45]:
```
16
```

You can also use dictionaries to pass keyword arguments with `**`.

In [46]:
```
pair = {'exp':4,'base':2}
int_exp(exp=pair['exp'],base=pair['base'])
```

Out[46]:
```
16
```

In [47]:
```
pair = {'exp':4,'base':2}
int_exp(**pair)
```

Out[47]:
```
16
```

Packing Arguments as Iterables

When defining a function you can pass variable number of arguments with `*`. All the remaining arguments will be stored to a tuple.

In [48]:
```
def int_exps(exp, *bases):
  for base in bases:
    print(base, "to the power of", exp, "is", int_exp(exp, base))
```

In [49]:
```
int_exps(2, 1, 2, 4, 66)
```

```
1 to the power of 2 is 1
2 to the power of 2 is 4
4 to the power of 2 is 16
66 to the power of 2 is 4356
```

Warning: When definig a function with variable number of arguments, the argument to be packed must be after fixed ones.

Similarly, you can pack the variable number of keyword arguments to a dictionary with `**`.

In [50]:
```
def define_car(car, **properties):
  for property in properties:
    print(property, "of", car, "is", properties[property])
```

In [51]:
```
define_car("mycar",manufacturer="Peugeot",model="308",color="black",engine_type="vti")
```

```
manufacturer of mycar is Peugeot
model of mycar is 308
color of mycar is black
engine_type of mycar is vti
```

Warning: When definig a function with variable number of arguments, the keyword arguments to be packed must be after fixed ones and positional ones.

In [52]:
```
def a(*x,**y):
  print("positional args:",x)
  print("keyword args:",y)

a(1,2,3,a=34,y=6)
```

```
positional args: (1, 2, 3)
keyword args: {'a': 34, 'y': 6}
```

Recursive Functions

A function is a sequence of other statements. Indeed a function can also call itself. This is sometimes useful instead of looping.

In [53]:
```
def factorial(n):
  "recursive factorial function"
  if n == 0:
    return 1
  return n * factorial(n-1)
```

In [54]:
```
factorial(125)
```

Out[54]:
```
188267717688892609974376770249160085759540364871492425887598231508353156331613598866882932889495923133646405445930057740630161919341380597818883457558547055524326375565007131770880000000000000000000000000000000
```

Nested Functions

Functions in Python can be nested i.e You can define a function inside another function definition. Since functions are some Python objects they can be returned by or passed as arguments to other functions.

In [55]:
```
def adder(a):
```

```
  "the function adder here defines another function which adds `a` to any given numbe
  "by doing this you can partially apply addition on numbers"
  def adder_a(b):
    return a+b
  return adder_a
```

In [56]:
```
f1 = adder(2)
f1(3)
```

Out[56]: 5

or equivalently:

In [57]:
```
adder(2)(3)
```

Out[57]: 5

Decoraters

Decoraters are special nested functions that convert a function to another function.

The following function checks the arguments of a function before executing, and raises an exception if any is negative. This can be used in our implementation of `gcd` function to prevent infinite loops.

In [58]:
```
def ensure_positive(func):
  def wrapper(*xs):
    if any(x < 0 for x in xs):
      raise ValueError("both arguments must be non-negative")
    return func(*xs)
  return wrapper
```

To apply a decorator to a function use `@` and decorator name before `def` statement line.

In [59]:
```
@ensure_positive
def gcd(a,b):
  "this function calculates the GCD of given numbers in ancient way."
  while a != b:
    if a > b:
      a -= b
    if a < b:
      b -= a

  return a
```

Noe `gcd` function is safer.

In [60]:
```
gcd(-2,12)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[60], line 1
----> 1 gcd(-2,12)

Cell In[58], line 4, in ensure_positive.<locals>.wrapper(*xs)
      2   def wrapper(*xs):
      3     if any(x < 0 for x in xs):
----> 4       raise ValueError("both arguments must be non-negative")
      5     return func(*xs)

ValueError: both arguments must be non-negative
```

Similarly we can apply `ensure_positive` decorator to `factorial` function as well:

In [ ]:
```
@ensure_positive
def factorial(n):
  "recursive factorial function"
  if n == 0:
    return 1
  return n * factorial(n-1)
```

In [ ]:
```
factorial(-5)
```

`all()` & `any()` Functions

Builtin `all` and `any` functions can be used to test the truth of tuples, lists, iterables.

In [ ]:
```
all((1 == 1., 1 < 2, 4 > 3, 3 != 4))
```

In [ ]:
```
all((0, 1, 2, 3))
```

The function `all` is equivalent to:

In [ ]:
```
def all(iterable):
    for element in iterable:
        if not element:
            return False
    return True
```

In [ ]:
```
any((1 == 1., 1 > 2, 4 < 3, 3 == 4))
```

In [ ]:
```
any((0, 0.0, "", (), [], {}))
```

The function `any` is equivalent to:

In [ ]:
```
def any(iterable):
    for element in iterable:
        if element:
            return True
    return False
```

Functional Programming

In Python functional programming means using functions as arguments to other functions. Some functions that uses other functions as atguments are:

`map` function

`map` function takes two arguments, first is a function and second is an iterable. And applies the function to each item of the iterable.

In [ ]:
```
for i in map(factorial,range(10)):
  print(i)
```

This is equivalent to:

In [ ]:
```
for i in range(10):
  print(factorial(i))
```

`filter` function

`filter` function selects some items from an iterable according to some function result.

In [ ]:
```
print(*filter(isodd,range(15)))
```

In [ ]:
```
def square(a):return a*a

sum(map(square, filter(isodd,range(10000))))
```

This is equivalent to:

In [ ]:
```
sum([square(i) for i in range(10000) if isodd(i)])
```

Functional Programming Exercise

Write a function that accepts two functions and an iterable. And tests whether their return values are equal or not for each item in the iterable.

In [ ]:
```
def compare(f1,f2,xs):
  for x in xs:
    if f1(x) != f2(x):
      return False
  return True
```

or:

In [ ]:
```
def compare(f1,f2,xs):
  return all(f1(x)==f2(x) for x in xs)
```

Lambda (Anonymuos) Functions

Instead of `def` statements, in Python you can define a function as an expression with `lambda` keyword. `lambda` functions are simple expressions written inline without a name.

For example:

In [ ]:
```
sum(map(lambda x:x*x*x, filter(lambda x:x&1==0,range(10000))))
```

Then you can assign it a name as well:

In [ ]:
```
compare = lambda f1,f2,xs: all(f1(x)==f2(x) for x in xs)
```

In [ ]:
```
compare(lambda x: x**2,lambda x: x**3,range(2))
```

`lambda` functions can also be nested.

In [ ]:
```
(lambda x: lambda y: x+y)(2)(3)
```

Warning: In `lambda` expressions we don't have `return`, `if`, `while`, `for`, `continue` and `break` statements.

In [ ]:
```
(lambda x,y=2,**z:(x,y,z))(1,c=2)
```